

排序

排序算法

冒泡排序

冒泡排序是一种简单的交换排序算法，通过重复遍历待排序的元素列，比较相邻的元素，如果顺序错误则交换它们，直到整个列排序完成。

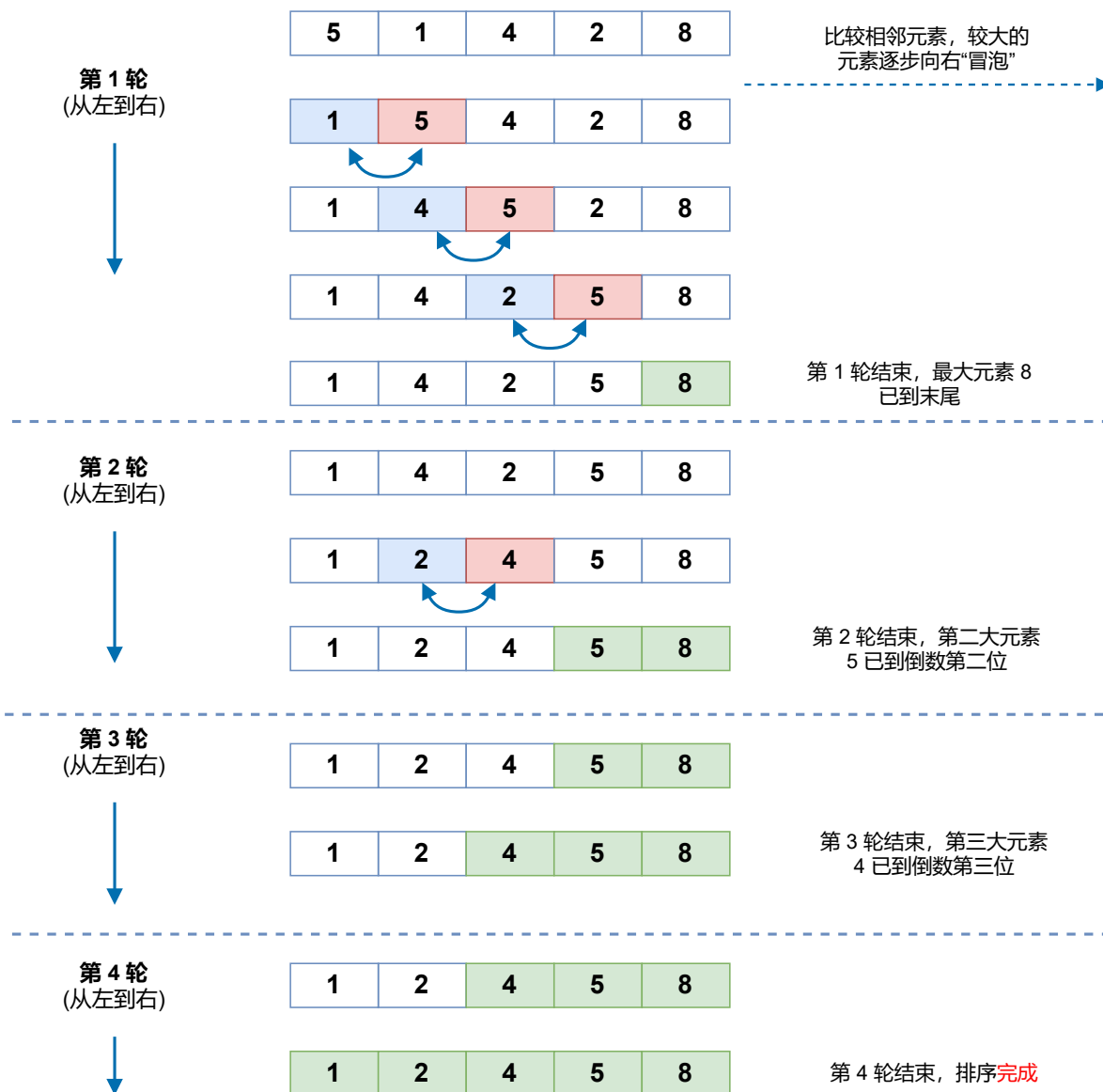
稳定

排序的**稳定性**是指：

对于关键字相同的两个元素，排序后它们的相对先后顺序是否保持不变。

冒泡排序

□ 未比较元素 □ 当前比较元素 □ 将要交换的元素 □ 已排序元素



原理

比较相邻元素

左 > 右 则 交换

每轮将最大元素“冒泡”到结尾



时间复杂度：

- 平均时间复杂度： $O(n^2)$
- 最坏时间复杂度： $O(n^2)$

```
1 void bubble_sort(int arr[], int n) {
2     for (int i = 0; i < n - 1; ++i) {
3         for (int j = 0; j < n - 1 - i; ++j) {
4             if (arr[j] > arr[j + 1]) {
5                 swap(arr[j], arr[j + 1]);
6             }
7         }
8     }
9 }
```

选择排序

不稳定

选择排序每次从待排序部分选择最小（或最大）元素，并与当前序列的第一个元素交换，直到序列完全排序。

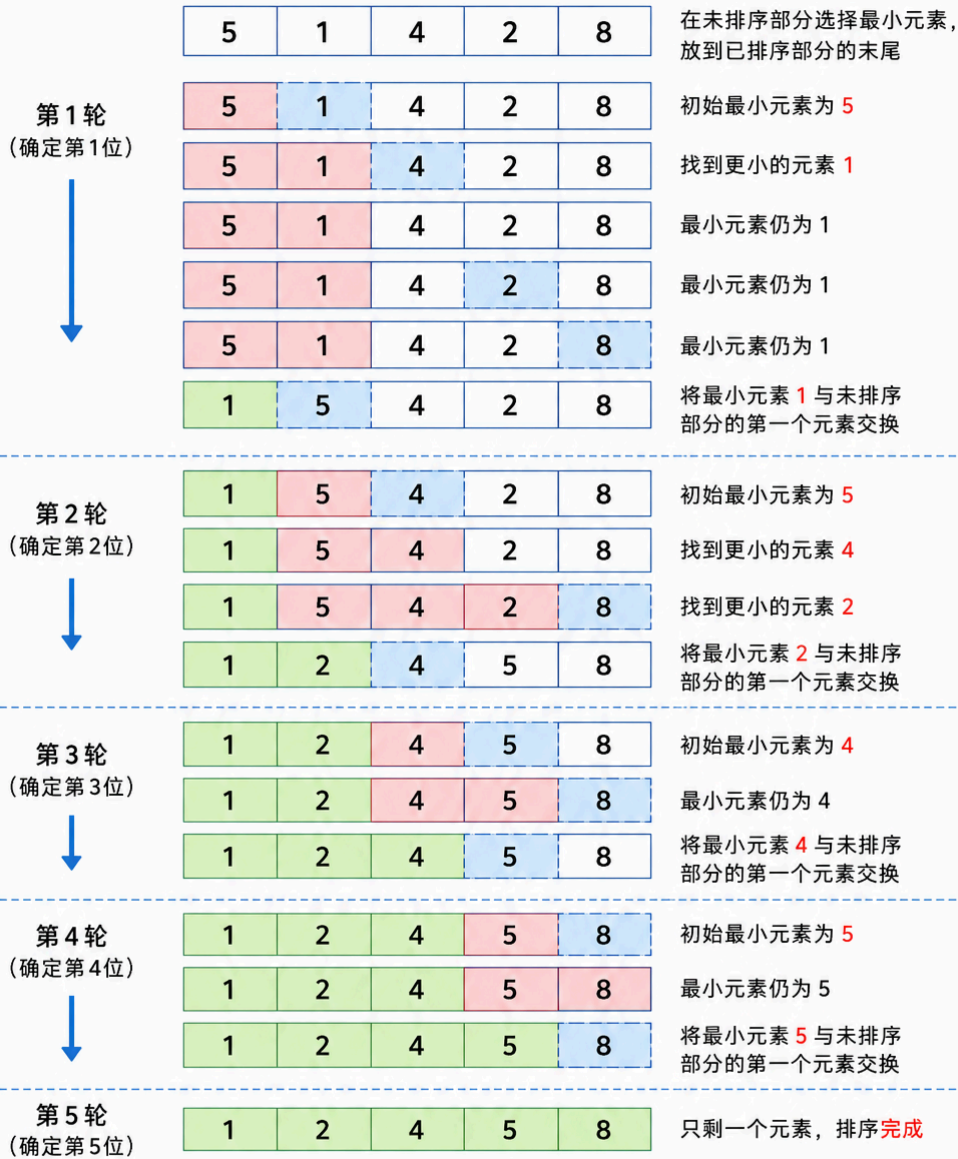
选择排序（升序）

未确定位置

当前扫描元素

当前最小元素

已确定位置



原理

在未排序部分
寻找最小元素

与未排序部分的
第一个元素交换

扩大已排序部分

重复以上过程



时间复杂度：

- 平均时间复杂度： $O(n^2)$
- 最坏时间复杂度： $O(n^2)$

```

1 void selection_sort(int arr[], int n) {
2     for (int i = 0; i < n - 1; ++i) {
3         int min_idx = i;
4         for (int j = i + 1; j < n; ++j) {
5             if (arr[j] < arr[min_idx]) {
6                 min_idx = j;
7             }
8         }
9         swap(arr[i], arr[min_idx]);
10    }
11 }

```

插入排序

稳定

插入排序通过逐步将待排序元素插入到已排序部分的适当位置，来完成排序。

时间复杂度：

- 平均时间复杂度： $O(n^2)$
- 最坏时间复杂度： $O(n^2)$

```

1 void insertion_sort(int arr[], int n) {
2     for (int i = 1; i < n; ++i) {
3         int key = arr[i];
4         int j = i - 1;
5         while (j >= 0 && arr[j] > key) {
6             arr[j + 1] = arr[j];
7             --j;
8         }
9         arr[j + 1] = key;
10    }
11 }

```

计数排序

稳定

计数排序是一种非比较型的排序算法，它通过统计每个元素出现的次数，并利用累加的频次值来确定元素的正确位置，从而实现排序。计数排序通常适用于范围较小的整数数据。

时间复杂度：

- 平均时间复杂度： $O(n + k)$
- 最坏时间复杂度： $O(n + k)$
 - 其中 n 是数组的长度， k 是数据的最大值。

计数排序的关键在于统计每个元素出现的频次，存储到一个计数数组中，然后根据计数数组来构造排好序的数组。

```

1  int* counting_sort(int* a, int n, int w) {
2      memset(cnt, 0, sizeof(cnt)); // 清空计数数组
3      for (int i = 1; i ≤ n; ++i) ++cnt[a[i]]; // 统计每个元素的出现次数
4      for (int i = 1; i ≤ w; ++i) cnt[i] += cnt[i - 1]; // 计算累加频次
5      for (int i = n; i ≥ 1; --i) b[cnt[a[i]]--] = a[i]; // 根据频次数组重新排序
6      return b; // 返回排序后的数组
7  }

```

STL排序及去重

时间复杂度： $O(n \log n)$ 。

升序排序

`sort(a,a+n);` 或 `sort(a+1,a+n+1);`

降序排序

`sort(a,a+n,greater<int>());` 或 `sort(a+1,a+n+1,greater<int>());`

数组去重

```

1  sort(a,a+n);
2  int m=unique(a,a+n)-a;//m表示去重后的数组长度

```

例题: P338. 【例58.1】明明的随机数

思路：

排完序调用去重函数即可。

代码：

```

1  #include<bits/stdc++.h>
2
3  using namespace std;
4
5  using i64=long long;
6
7  int a[110];
8  void Showball(){
9      int n;
10     cin>>n;
11     for(int i=0;i<n;i++) cin>>a[i];
12
13     sort(a,a+n);
14
15     int m=unique(a,a+n)-a;
16     cout<<m<<"\n";
17     for(int i=0;i<m;i++) cout<<a[i]<<" ";
18 }
19 int main(){
20     ios::sync_with_stdio(false);
21     cin.tie(nullptr);
22     int t=1;

```

```

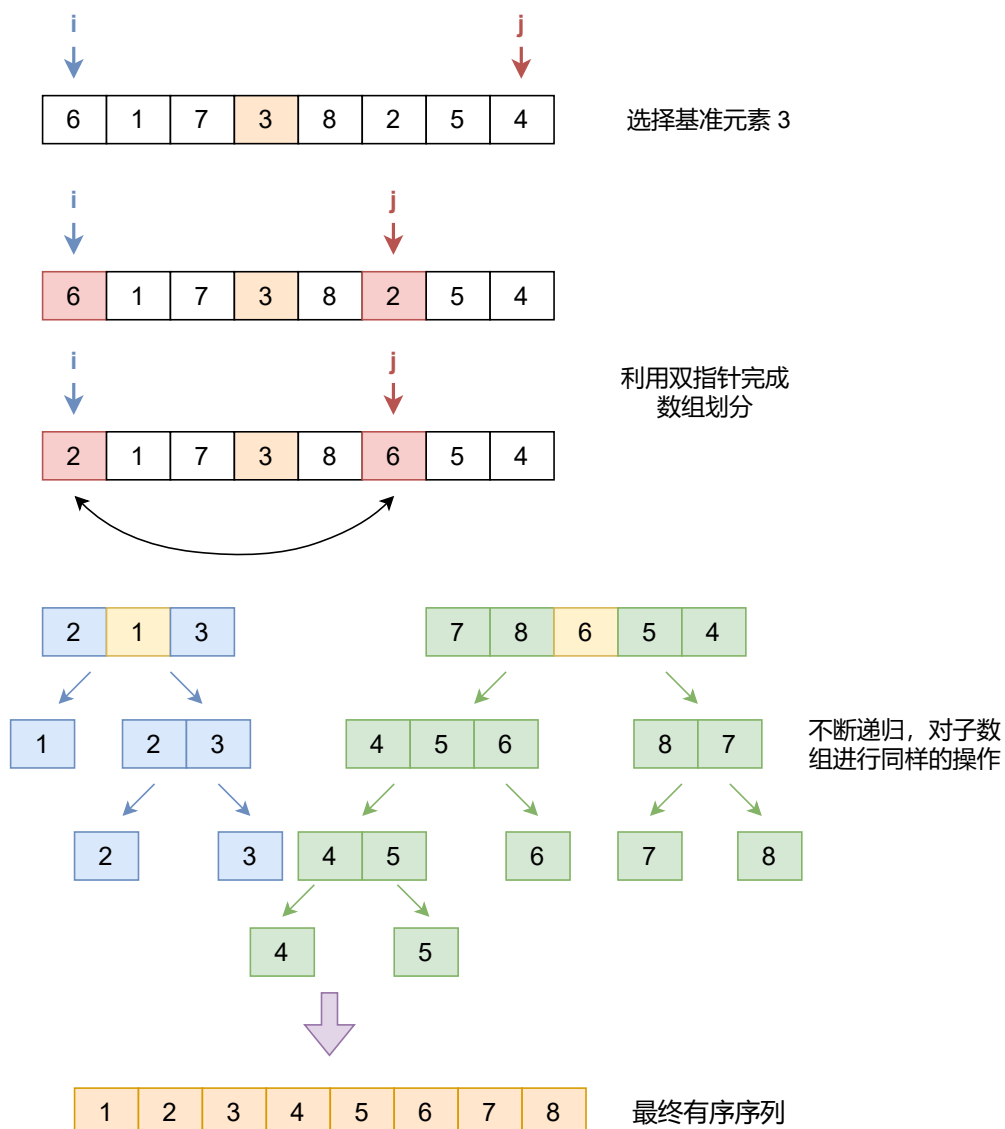
23     //cin>>t;
24
25     while(t--){
26         Showball();
27     }
28
29     return 0;
30 }

```

快速排序

不稳定

快速排序是一种基于分治思想的高效排序算法。它通过选择一个基准元素，将数组分为两部分，使得左边部分的元素都小于等于基准元素，右边部分的元素都大于等于基准元素，然后分别对左右两部分递归地进行排序，最终实现整个数组的有序排列。



时间复杂度:

- 平均时间复杂度: $O(n \log n)$
- 最坏时间复杂度: $O(n^2)$ (当数组已经接近有序时, 递归树退化为链式结构)

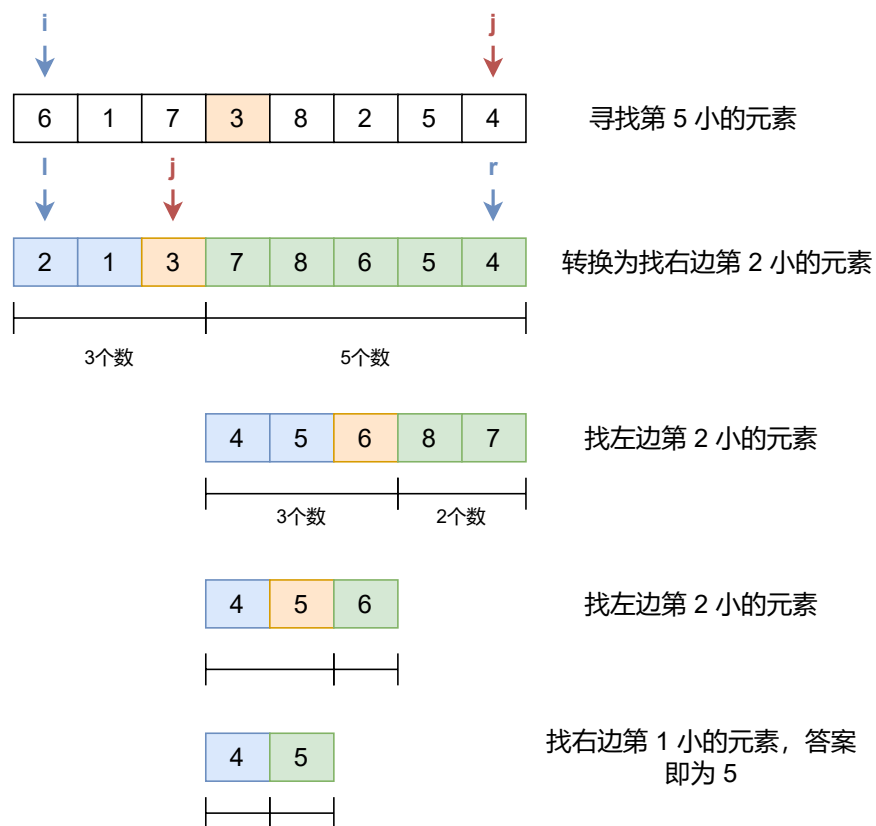
```

1  void quick_sort(int q[], int l, int r) {
2      if (l >= r) return;
3
4      int i = l - 1, j = r + 1, x = q[l + r >> 1];
5      while (i < j) {
6          do i++; while (q[i] < x); // 双指针
7          do j--; while (q[j] > x);
8          if (i < j) swap(q[i], q[j]);
9      }
10
11     quick_sort(q, l, j);
12     quick_sort(q, j + 1, r);
13 }

```

第k个数（快速选择算法）

快速选择算法是基于快速排序的思想，用来查找数组中的第 k 小元素，时间复杂度为 $O(n)$ 。



时间复杂度：

- 平均时间复杂度： $O(n)$

```

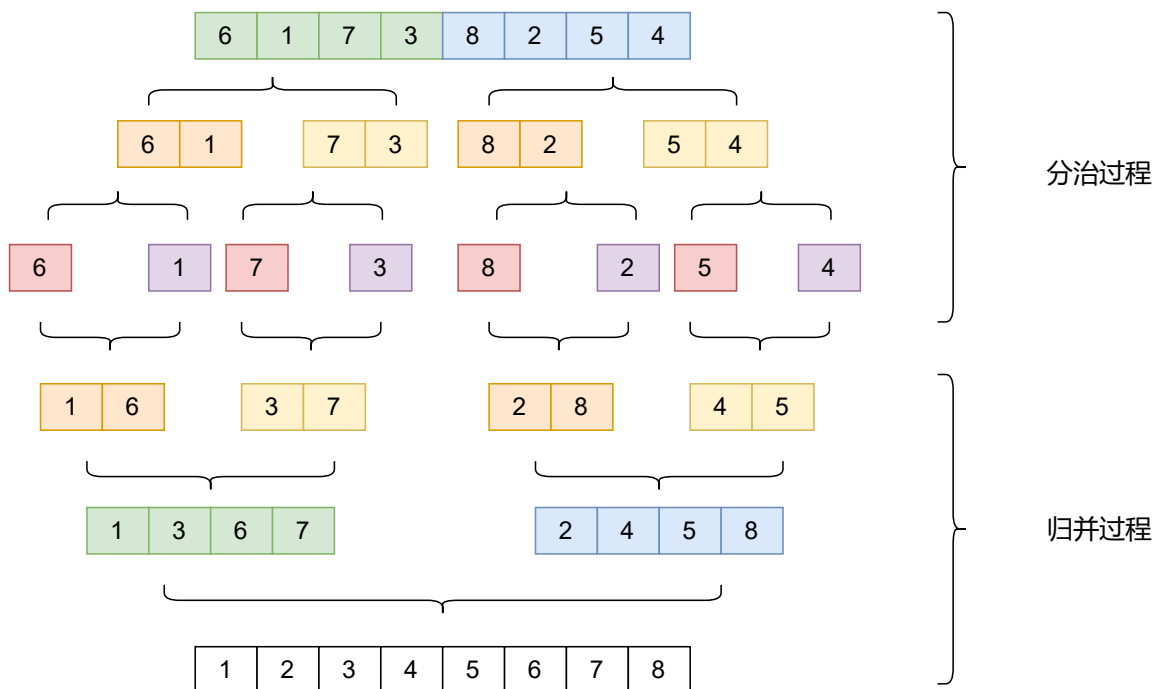
1  int quick_select(int q[], int l, int r, int k) {
2      if (l ≥ r) return q[l];
3      int i = l - 1, j = r + 1, x = q[l + r >> 1];
4      while (i < j) {
5          do i++; while (q[i] < x);
6          do j--; while (q[j] > x);
7          if (i < j) swap(q[i], q[j]);
8      }
9
10     int sl = j - l + 1;
11     if (k ≤ sl) return quick_select(q, l, j, k);
12     else return quick_select(q, j + 1, r, k - sl);
13 }

```

归并排序

稳定

归并排序是一种稳定的排序算法，同样基于分治思想。它将一个数组分成两个子数组，分别对这两个子数组进行排序，然后将排好序的子数组合并成一个有序的数组。



时间复杂度：

- 平均时间复杂度： $O(n \log n)$
- 最坏时间复杂度： $O(n \log n)$

```

1  void merge_sort(int q[], int l, int r) {
2      if (l ≥ r) return;
3
4      int mid = l + r >> 1;
5      merge_sort(q, l, mid);
6      merge_sort(q, mid + 1, r);
7
8      int k = 0, i = l, j = mid + 1;
9      while (i ≤ mid && j ≤ r) {
10         if (q[i] ≤ q[j]) tmp[k++] = q[i++];

```

```

11         else tmp[k++] = q[j++];
12     }
13     while (i ≤ mid) tmp[k++] = q[i++];
14     while (j ≤ r) tmp[k++] = q[j++];
15
16     for (i = l, j = 0; i ≤ r; i++, j++) q[i] = tmp[j];
17 }

```

归并排序求逆序对的数量

逆序对是数组中一对“前面的数比后面的数大”的元素。

对于数组 a ，若存在下标 $i < j$ ，且 $a[i] > a[j]$ 。

那么 $(a[i], a[j])$ 就构成一个逆序对。

容易发现，**冒泡排序** 的交换次数就是逆序对的数量。所以，我们可以使用冒泡排序 $O(n^2)$ 求出逆序对的数量。

归并排序可以在排序的同时统计逆序对的数量。

时间复杂度: $O(n \log n)$ 。

```

1  int merge_sort(int q[], int l, int r) {
2      if (l ≥ r) return 0;
3
4      int mid = l + r >> 1;
5      int res = merge_sort(q, l, mid) + merge_sort(q, mid + 1, r);
6
7      int k = 0, i = l, j = mid + 1;
8      while (i ≤ mid && j ≤ r) {
9          if (q[i] ≤ q[j]) tmp[k++] = q[i++];
10         else {
11             res += mid - i + 1;
12             tmp[k++] = q[j++];
13         }
14     }
15     while (i ≤ mid) tmp[k++] = q[i++];
16     while (j ≤ r) tmp[k++] = q[j++];
17
18     for (i = l, j = 0; i ≤ r; i++, j++) q[i] = tmp[j];
19
20     return res;
21 }

```

结构体重载排序

可以通过重载 `<` 运算符或者定义自定义的比较函数来对结构体进行排序。

重载运算符

```

1 //按照分数从大到小排序，分数相同，按照学号从小到大排序
2 struct Student {
3     int score;
4     int id;
5     string name;
6     bool operator < (const Student &s) const {
7         if(score  $\neq$  s.score) return score > s.score;
8         else return id < s.id;
9     }
10 };

```

自定义比较函数

```

1 bool cmp(Student x, Student y){
2     if(x.score  $\neq$  y.score) return x.score > y.score;
3     else return x.id < y.id;
4 }

```

例题:P1093. [NOIP 2007 普及组] 奖学金

思路：

创建结构体，为了方便，我们存储每个学生的学号，语数英成绩和总分。读入后，记录学号和总分。然后按照题目规则写出 `cmp` 函数即可。

代码：

```

1 #include<bits/stdc++.h>
2
3 using namespace std;
4
5 using i64=long long;
6
7 struct node{
8     int id;
9     int sum;
10    int ch,ma,en;
11 }a[310];
12
13 bool cmp(node x,node y){
14     if(x.sum $\neq$ y.sum) return x.sum>y.sum;
15     if(x.ch $\neq$ y.ch) return x.ch>y.ch;
16     return x.id<y.id;
17 }
18
19 void Showball(){
20     int n;
21     cin>>n;
22     for(int i=0;i<n;i++){
23         cin>>a[i].ch>>a[i].ma>>a[i].en;
24         a[i].sum=a[i].ch+a[i].ma+a[i].en;
25         a[i].id=i+1;
26     }

```

```
27
28     sort(a,a+n,cmp);
29
30     for(int i=0;i<5;i++){
31         cout<<a[i].id<<" "<<a[i].sum<<"\n";
32     }
33
34 }
35 int main(){
36     ios::sync_with_stdio(false);
37     cin.tie(nullptr);
38     int t=1;
39     //cin>>t;
40
41     while(t--){
42         Showball();
43     }
44
45     return 0;
46 }
```